

Documento de Requisitos

Sistema de Cadastro de Alunos (SISTEMA-ALUNOS) Migração do programa Java para Python

1. Visão geral

Sistema para cadastrar alunos e suas notas, listar os dados cadastrados e buscar a nota de um aluno pelo nome. O programa original em **Java** (`SistemaAlunos.java`) é a base desta especificação, que o descreve, propõe sua reescrita em **Python** e uma evolução com persistência em banco de dados.

1.1 Comportamento do programa Java original

Programa de console com menu (1 - Adicionar aluno, 2 - Listar alunos, 3 - Buscar nota, 0 - Sair). Os dados ficam apenas em memória: um `HashMap<String, Double>` guarda as notas e um `ArrayList<String>` guarda a ordem dos nomes. Ao encerrar, tudo é perdido.

1.2 Limitações observadas no código Java (corrigidas na versão Python)

Limitação no Java	Consequência	Tratamento no Python
Nota lida com <code>nextDouble()</code> sem validar faixa	Aceita notas como -5 ou 100	Validação de 0 a 10 (RF06)
Texto no lugar de número (opção ou nota)	Lança <code>InputMismatchException</code> e encerra o programa	Novo pedido de entrada, sem encerrar (RNF02)
Nota com vírgula depende do idioma do sistema (Locale)	Pode falhar em máquinas com Locale diferente	Aceita ponto ou vírgula (RF09)
Nome repetido: <code>put</code> sobrescreve a nota, mas o nome entra de novo na lista	Aluno aparece duplicado na listagem	Nome único: nova nota atualiza o aluno (RN04)
Nome vazio é aceito	Cadastro de aluno sem nome	Nome obrigatório (RN03)
Dados apenas em memória	Perda dos dados ao sair	Persistência em banco (RF12, evolução)

2. Requisitos funcionais

ID	Requisito
RF01	O sistema deve exibir um menu em console com as opções: adicionar aluno, listar alunos, buscar nota e sair.
RF02	O sistema deve permitir cadastrar um aluno informando nome e nota final.
RF03	O sistema deve listar todos os alunos cadastrados com suas notas, na ordem de cadastro, ou informar que não há alunos.
RF04	O sistema deve buscar a nota de um aluno pelo nome, informando quando o aluno não for encontrado.
RF05	O sistema deve repetir o menu até o usuário escolher sair e informar opção inválida quando for o caso.
RF06	O sistema deve validar que a nota esteja entre 0 e 10.
RF07	O sistema deve exigir que o nome do aluno não seja vazio.
RF08	O sistema deve calcular e exibir a média da turma (evolução: não existe no Java).
RF09	O sistema deve aceitar notas decimais (ponto ou vírgula).
RF10	(evolução) O sistema deve permitir editar e excluir um aluno cadastrado.
RF11	(evolução) O sistema deve indicar a situação do aluno (aprovado/reprovado) com base em uma nota mínima configurável.
RF12	(evolução) O sistema deve persistir os dados em banco de dados.

3. Requisitos não funcionais

ID	Categoria	Requisito
RNF01	Usabilidade	Mensagens de entrada e erro devem ser claras e em português.
RNF02	Confiabilidade	Entradas inválidas (texto no lugar de número, nota fora da faixa, opção inexistente) não devem encerrar o programa.
RNF03	Desempenho	A listagem, a busca e o cálculo da média devem responder em menos de 1 segundo para até 1.000 alunos.
RNF04	Manutenibilidade	O código deve ser modular (classes separadas por responsabilidade) e seguir a PEP 8.
RNF05	Portabilidade	O sistema deve rodar em Windows, Linux e macOS com Python 3.10 ou superior, sem depender de JVM.
RNF06	Precisão	Notas e média devem ser exibidas com 2 casas decimais.
RNF07 (evolução)	Segurança	Os dados persistidos devem ter acesso restrito e backup periódico.
RNF08	Testabilidade	A lógica de cálculo e validação deve ser coberta por testes unitários.

4. Regras de negócio

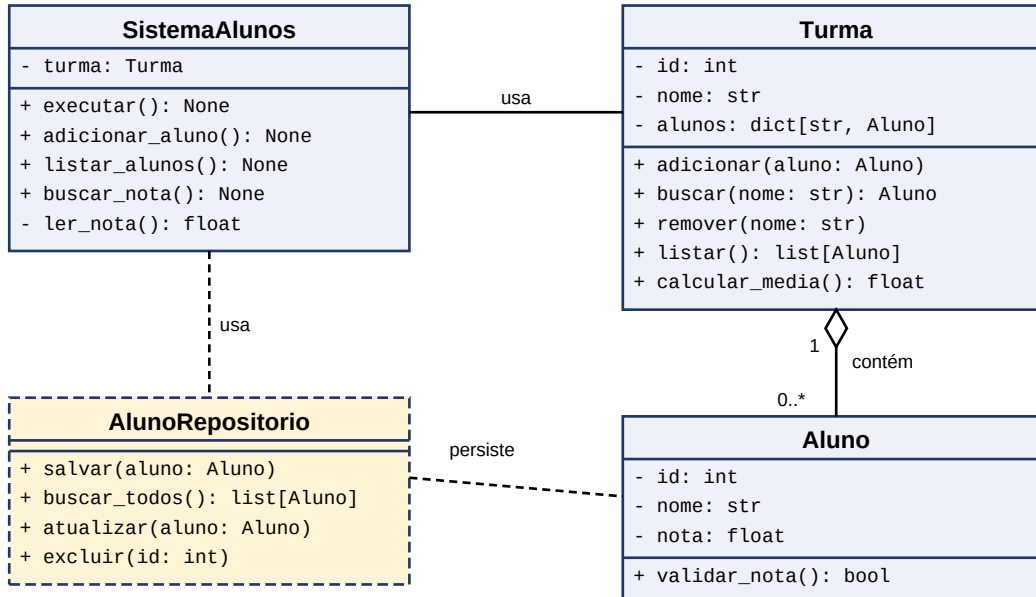
RN01: média = soma das notas ÷ quantidade de alunos (0 quando não há alunos).

RN02: nota válida está no intervalo de 0 a 10.

RN03: nome do aluno é obrigatório e não pode ser vazio.

RN04: o nome identifica o aluno na turma; cadastrar um nome já existente atualiza a nota em vez de duplicar o registro.

5. Diagrama de classes



Java original

```
static HashMap notas
static ArrayList
    listaNomes
main() + switch
    adicionarAluno()
    listarAlunos()
    buscarNota()
```

Python

```
Turma (dict)
    (substitui os 2
     campos estáticos)
executar() + dict
menu: SistemaAlunos
```

Caixa tracejada = evolução (não existe no programa Java original).

Turma contém zero ou mais **Alunos** (agregação). **SistemaAlunos** usa **Turma** e **AlunoRepositorio**; **AlunoRepositorio** persiste objetos **Aluno**. No Java original tudo estava em uma única classe, com dois campos estáticos (**notas** e **listaNomes**); no Python eles passam a ser a classe **Turma**, e o menu fica em **SistemaAlunos**.

6. Entidades e tabelas possíveis

Versão mínima (cobrir o programa atual)

Tabela	Campos principais
TURMA	id (PK), nome, ano_letivo
ALUNO	id (PK), nome, turma_id (FK para TURMA)
NOTA	id (PK), aluno_id (FK para ALUNO), valor (DECIMAL(4,2)), data_lancamento

Versão ampliada (se o sistema crescer)

Tabela	Campos principais
DISCIPLINA	id, nome, carga_horaria
PROFESSOR	id, nome, email
MATRICULA	id, aluno_id, turma_id, data_matricula
AVALIACAO	id, disciplina_id, descrição, peso, data
NOTA	id, matricula_id, avaliacao_id, valor

Relacionamentos principais: Turma 1:N Aluno (versão mínima); Aluno N:N Turma por meio de Matrícula (versão ampliada); Aluno 1:N Nota. Recomenda-se uma restrição CHECK (valor BETWEEN 0 AND 10) na coluna da nota, para garantir a RN02 também no banco. No Java original, a entidade ALUNO corresponde a chave do HashMap (nome) e a NOTA ao seu valor.

7. Bancos de dados indicados

Banco	Quando usar
SQLite	Melhor escolha para este projeto: já vem com o Python (módulo sqlite3), não exige instalação e serve bem para estudo e uso local.
PostgreSQL	Se virar um sistema web com vários usuários simultâneos; é robusto, gratuito e têm ótimo suporte a restrições e integridade.
MySQL / MariaDB	Alternativa muito comum em hospedagens e sistemas web; no mundo Java é a combinação clássica com JDBC.
SQL Server / Oracle	Faz sentido em ambientes corporativos ou legados que já usam essas plataformas.

Como os dados são tabulares e têm relacionamentos claros (aluno, turma, notas), bancos relacionais são o caminho natural. Bancos NoSQL, como o MongoDB, não trazem vantagem neste cenário.

8. Mapeamento Java para Python

Java	Python
Scanner sc = new Scanner(System.in)	input()
HashMap<String, Double> notas + ArrayList<String> listaNomes	dict[str, Aluno] (dict preserva a ordem de inserção)
switch (opção) { case 1: ... }	dicionário de ações (ou match/case)
do { ... } while (opção != 0)	while True: com break
notas.containsKey(nome)	nome in turma / dict.get(nome)
System.out.println(...)	print(f"...")
static (métodos e campos globais)	classes e instâncias (self)